HW5: Lighting Models — Comparison Report

Daniel Oni | CS 7382 Computer Graphics | Spring 2026

Screenshots: Gouraud (Per-Vertex) vs. Cook-Torrance (PBR)

[Fig 1: Gouraud screenshot]	[Fig 2: Cook-Torrance screenshot]

Fig 1 (left): Gouraud shading — bright golden diffuse with visible specular glow, 4 subdivisions. Fig 2 (right): Cook-Torrance (PBR) — dramatically darker with subtle specular, same angle and subdivisions.

What I See (Visual Differences)

This is honestly the most striking comparison. The Gouraud sphere is bright — you get that warm golden-orange color across the lit side with a smooth gradient fading into the dark side. The specular region blends into the diffuse and kind of spreads out because the per-vertex pow() calculation gets linearly interpolated, so the highlight is soft and broad. The whole lit portion of the sphere feels evenly warm. The Cook-Torrance sphere is a completely different vibe. It's overwhelmingly dark — almost the entire sphere is in that deep purple-brown range with only a faint lighter patch near the top-left where the light is hitting most directly. This is energy conservation doing its thing: the line `kD = (1.0 - F)` in the shader subtracts the Fresnel-reflected energy from the diffuse component, so what doesn't get reflected as specular is all you see as diffuse color. Since the Fresnel term is relatively high across most of the surface, the diffuse contribution gets heavily reduced. The specular highlight itself is tight and concentrated (thanks to GGX distribution) but because the roughness derived from shininess=20 is moderate, the highlight stays subtle rather than a bright pinpoint. The user can increase subdivisions or rotate with the theta/phi buttons, and the effect stays consistent — Cook-Torrance always looks darker and more physically grounded compared to Gouraud's brighter, more traditional look.

How the Code Produces These Effects

Gouraud (vertex shader computes everything):

```
float Kd = max(dot(L,N), 0.0); vec4 diffuse = Kd*uDiffuseProduct;
float Ks = pow(max(dot(N,H), 0.0), uShininess); vec4 specular = Ks*uSpecularProduct;
vColor = ambient + diffuse + specular;  // final color per vertex, then interpolated
```

The fragment shader is just `fColor = vColor`. Diffuse and specular are simply added together with no energy budget — the surface can technically reflect more light than it receives. This is why Gouraud looks so bright: diffuse + specular stack on top of each other freely. Pattern matches *shadedSphere1.html* from our course examples.

Cook-Torrance (per-fragment PBR microfacet BRDF):

```
float D = distributionGGX(NN, H, roughness);  // GGX normal distribution
float G = geometrySmith(NN, VV, LL, roughness); // Smith geometry occlusion
vec3 F = fresnelSchlick(max(dot(H,VV),0.0), F0); // Schlick Fresnel
vec3 spec = (D*G*F) / max(4.0*NdotV*NdotL, 0.001);
vec3 kD = (1.0 - F);  // ENERGY CONSERVATION - key difference
vec3 Lo = (kD*uDiffuseProduct.rgb/PI + spec) * NdotL;
```

Three things explain the dark appearance. First, `kD = (1.0 - F)` enforces energy conservation — specular reflection energy is subtracted from diffuse, so total output never exceeds input. Second, diffuse is divided by PI (`uDiffuseProduct.rgb/PI`) for physically correct Lambertian normalization, which further reduces brightness. Third, the GGX distribution concentrates the specular into a narrow peak rather than spreading it broadly like Blinn-Phong's pow(). The Smith geometry term (`geometrySmith`) also attenuates light at grazing angles where microfacets shadow each other. Together these produce a much more realistic but darker result than the traditional additive Gouraud model.

Program switching: Three GPU program objects are created via `initShaders()`, each with its own VAO (pattern from *Ex_MultipleShadersVAO.html*). The dropdown triggers `gl.useProgram()` + `gl.bindVertexArray()` to swap at render time. All programs share the same subdivided sphere VBOs.